

# Linear Algebra With Unity

\*\*\* 선형대수학 학습을 통해 Unity의 실력향상을 기대할 수 있습니다. \*\*\*

\*\*\* 선형대수학 학습을 통해 컴퓨터 그래픽의 실력향상을 기대할 수 있습니다.\*\*\*

2021/02/20 ~

박지원

# 목차

## <0강\_선형대수학 개요>

)

## <1강\_Vector>

### 1. Vector의 정의

- 객체의 크기뿐만 아니라 방향 까지 표기하는 것을 Vector라고 합니다.

ex)  $\text{vector}(1,1) = (1,1)$ 방향으로 루트2의 크기를 갖는 객체.

- Vector는 화살표로 표현합니다.

- 물리학에서는 주로 힘을 표현하고, Unity에서는 위치, 방향, 속도를 Vector로 표기합니다.

### 2. Vector의 절대 좌표와 상대 좌표

<절대 좌표>

= 원점(=영벡터 =  $(0,0,0)$ )에 대하여 방향과 크기를 나타낸 Vector

<상대 좌표>

= 자신이 아닌 임의의 다른 벡터(ex :  $(4,4,4)$ )에 대하여 방향과 크기를 나타낸 Vector

#### 1) Vector 1개에서의 개념

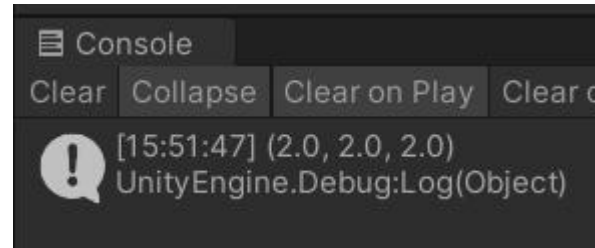
```
gameObject.transform.position = new Vector3(2, 2, 2);
```

- `gameObject`의 `position`을  $(2,2,2)$ 로 생각하여 스칼라로 볼 수 있으나, 이를 엄밀히 따져보면 원점을 기준으로 방향 $(2,2,2)$ 과 크기(루트12)를 나타는 벡터로 생각해야 합니다.

- Vector가 1개 주어지면 언제나 원점을 기준으로 벡터를 표기하기 때문에 절대 좌표와 상대 좌표가 항상 동일합니다.

## 2) Vector 2개에서의 개념

```
Vector3 v1 = new Vector3(2, 2, 2);  
Vector3 v2 = new Vector3(4, 4, 4);  
Debug.Log(v2 - v1);
```



- 정점 V2의 절대 좌표는 (4,4,4)이지만 , V1에 대해 상대 좌표를 구하면 (4,4,4) - (2,2,2)로 (2,2,2)의 Vector를 나타냅니다.

## 3. Vector의 크기

$\mathbb{R}^n$ 의 벡터  $\mathbf{x} = (x_1, x_2, \dots, x_n)$ 에 대하여

$$\|\mathbf{x}\| = \sqrt{x_1^2 + x_2^2 + \dots + x_n^2}$$

을  $\mathbf{x}$ 의 **노름(norm, length, magnitude)**이라 한다.

위의 정의에서  $\|\mathbf{x}\|$ 는 원점에서 점  $P(x_1, x_2, \dots, x_n)$ 에 이르는 거리로 정의됨을 의미한다. 따라서  $\mathbb{R}^n$ 의 두 벡터  $\mathbf{x} = (x_1, x_2, \dots, x_n)$ ,  $\mathbf{y} = (y_1, y_2, \dots, y_n)$ 에 대하여  $\|\mathbf{x} - \mathbf{y}\|$ 는 두 점  $P(x_1, x_2, \dots, x_n)$ 와  $Q(y_1, y_2, \dots, y_n)$  사이의 **거리**로 정의한다. 즉,

$$\|\mathbf{x} - \mathbf{y}\| = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + \dots + (x_n - y_n)^2}$$

```
gameObject.transform.position.magnitude
```

#### 4. Vector의 내적과 각도

##### 1) 내적

$\mathbb{R}^n$ 의 벡터  $x = (x_1, x_2, \dots, x_n)$ ,  $y = (y_1, y_2, \dots, y_n)$ 에 대하여 실수

$$x_1y_1 + x_2y_2 + \dots + x_ny_n$$

을  $x$ 와  $y$ 의 **내적**(Euclidean inner product, dot product)이라 하고  $x \cdot y$ 로 나타낸다. 즉

$$x \cdot y = x_1y_1 + x_2y_2 + \dots + x_ny_n$$

- 내적은 벡터에서의 곱을 의미합니다.
- Unity에서는 내적의 값이 0이면 두 벡터는 수직인 성질을 검사할 때 사용합니다.
- **내적을 이용하면 두 벡터의 각도를 구할 수 있습니다. 하지만 더욱 간단한 method가 존재합니다.**

## 2) 두 벡터의 각도

$\mathbb{R}^n$ 의 벡터  $x, y$ 에 대하여

$$x \cdot y = \|x\| \|y\| \cos \theta, 0 \leq \theta \leq \pi$$

인  $\theta$ 를  $x$ 와  $y$ 가 이루는 **각**(angle, 사잇각)이라 한다.

## 5. GameObject 회전

- Euler Angle을 쓰면 물체의 회전이 친숙하지만 이를 사용하면 오류가 발생하므로 Unity는 Quaternion이라는 개념을 사용합니다.
- Quaternion은 4개의 인자를 갖는 좌표계이며 이해하기는 어렵지만 Unity내부에서 Quaternion을 Euler처럼 사용할 수 있게 method를 구현했습

니다.

#### 1) 1회성 회전

```
transform.rotation = Quaternion.Euler(10,20,30);
```

- 위의 method를 사용해서 (x축으로 회전 , y축으로 회전 , z축으로 회전) 구현할 수 있습니다.

#### 2) 연속적인 회전

```
private void Update()
{
    const float CONSTANT = 3f;
    transform.Rotate(0, Time.deltaTime * CONSTANT, 0);
}
```

#### 3) 시계 OR 공전 같은 회전

## RotateAround(Vector3 point, Vector3 axis, float angle)

```
transform.RotateAround(new Vector3(0,0,0), Vector3.down , 1 );  
//2번째 인자는 단위벡터로 나타내는 좌표축.
```

|                                  |                                                                                                        |
|----------------------------------|--------------------------------------------------------------------------------------------------------|
| <a href="#">back</a>             | Shorthand for writing Vector3(0, 0, -1).                                                               |
| <a href="#">down</a>             | Shorthand for writing Vector3(0, -1, 0).                                                               |
| <a href="#">forward</a>          | Shorthand for writing Vector3(0, 0, 1).                                                                |
| <a href="#">left</a>             | Shorthand for writing Vector3(-1, 0, 0).                                                               |
| <a href="#">negativeInfinity</a> | Shorthand for writing Vector3(float.NegativeInfinity, float.NegativeInfinity, float.NegativeInfinity). |
| <a href="#">one</a>              | Shorthand for writing Vector3(1, 1, 1).                                                                |
| <a href="#">positiveInfinity</a> | Shorthand for writing Vector3(float.PositiveInfinity, float.PositiveInfinity, float.PositiveInfinity). |
| <a href="#">right</a>            | Shorthand for writing Vector3(1, 0, 0).                                                                |
| <a href="#">up</a>               | Shorthand for writing Vector3(0, 1, 0).                                                                |
| <a href="#">zero</a>             | Shorthand for writing Vector3(0, 0, 0).                                                                |

### <매개변수>

- 태양과 지구의 공전을 예로 들겠습니다.

- ① point : 태양의 좌표
- ② axis(회전축) : 지구가 공전할 때 기준으로 삼는 좌표축 (위의 method를 이용합니다.)
- ③ angle : 1회 공전하는 각도

## 6. Vector의 정규화

- Vector에서 크기는 무시하고 방향 정보만 필요할 때가 많습니다. 그 때 Vector의 크기가 1이면 편리하므로 Vector의 크기를 1로 만들어 버립니다. 이를 Vector의 정규화라고 합니다.

```
Vector3 new_vector = transform.position.normalized;  
  
float length = transform.position.magnitude;  
Vector3 new_vector_2 = transform.position / length;
```

new\_vector가 method를 사용해서 정규화를 표현한 것입니다.

new\_vector2는 일반적으로 정규화를 구하는 방식 입니다.

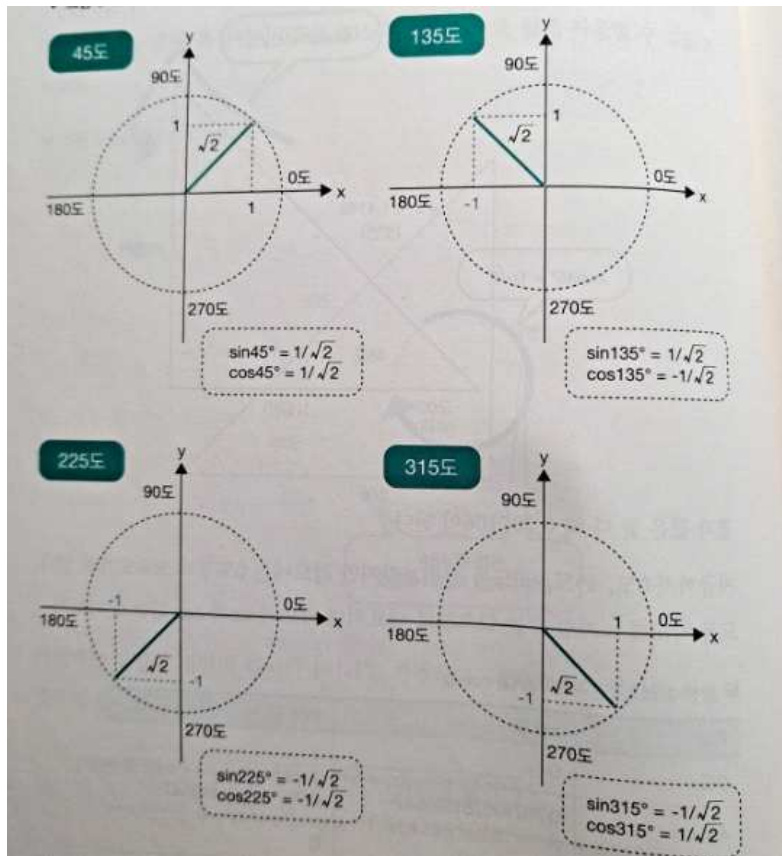
<주의 사항>

- 정규화한 vector의 크기를 Unity에서 출력해보면 언제나 1이 나옵니다.
- 하지만 좌표를 출력하고  $x\text{좌표}^2 + y\text{좌표}^2 + z\text{좌표}^2$ 를 계산하면 1이 아닌 경우가 종종 발생합니다. 예를 들어 (1,1,1)를 정규화 시키면 (0.36,0.36,0.36)이 나오며 이 좌표의 크기를 method로 구하면 1이 나오지만 실제로 계산해보면 1을 넘어섭니다. 그러나 이는 오류가 아니며 , 어떤 수를 제공해서 0.33333을 만드는 것이 사실상 불가능하고 이미 정규화를 구하는 것 자체가 크기를 무시하는 것이기 때문에 x좌표 , y좌표 , z좌표의 비율을 유지하며 근사한 값으로 출력하는 것 같습니다.

## <2강\_삼각함수>

### 1. 기초 개념

- CPU는 각도에서 도를 사용하지 않고 라디안(파이 표기법. ex :  $60^\circ = \pi/3$ )을 이용해서 나타냅니다.
- 삼각함수 공식은 method를 이용해서 구할 수 있으므로 외우지 않아도 되지만 성질과 관련된 공식정도는 이해를 하고 있으면 좋습니다.



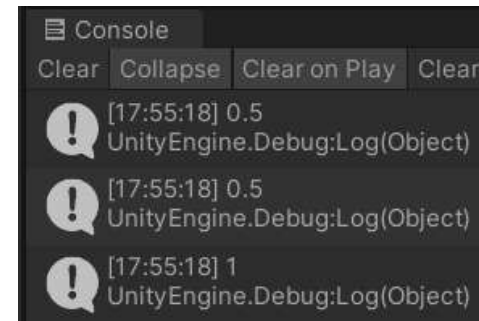
- 빗변은 언제나 양수입니다.
- $\sin$ 의 양의 영역 : 1,2 사분면
- $\sin$ 의 음의 영역 : 3,4 사분면
- $\cos$ 의 양의 영역 : 1,4 사분면
- $\cos$ 의 음의 영역 : 2,3 사분면

## 2. sin , cos , tan 값 구하기

= 각도(라디안 표기법)를 매개변수로 하여 삼각 함수의 결과 값을 계산하는 method

- Math.PI : 라이브러리에서 만든 파이 상수

```
Debug.Log(Mathf.Sin(Mathf.PI / 6)); //30도  
Debug.Log(Mathf.Cos(Mathf.PI / 3)); //60도  
Debug.Log(Mathf.Tan(Mathf.PI / 4)); //45도
```



## 3. 역삼각함수

= 삼각 함수의 결과 값을 매개변수로 하여 각도를 계산하는 method

- 기초공학수학에서 배운 삼각함수의 역함수인 역삼각함수를 이용하며 이들의 명칭에는 **Arc**가 들어갑니다.

- C#에서 주어주는 역삼각함수는 Arc를 a로 줄여서 sin , cos , tan에 붙입니다.

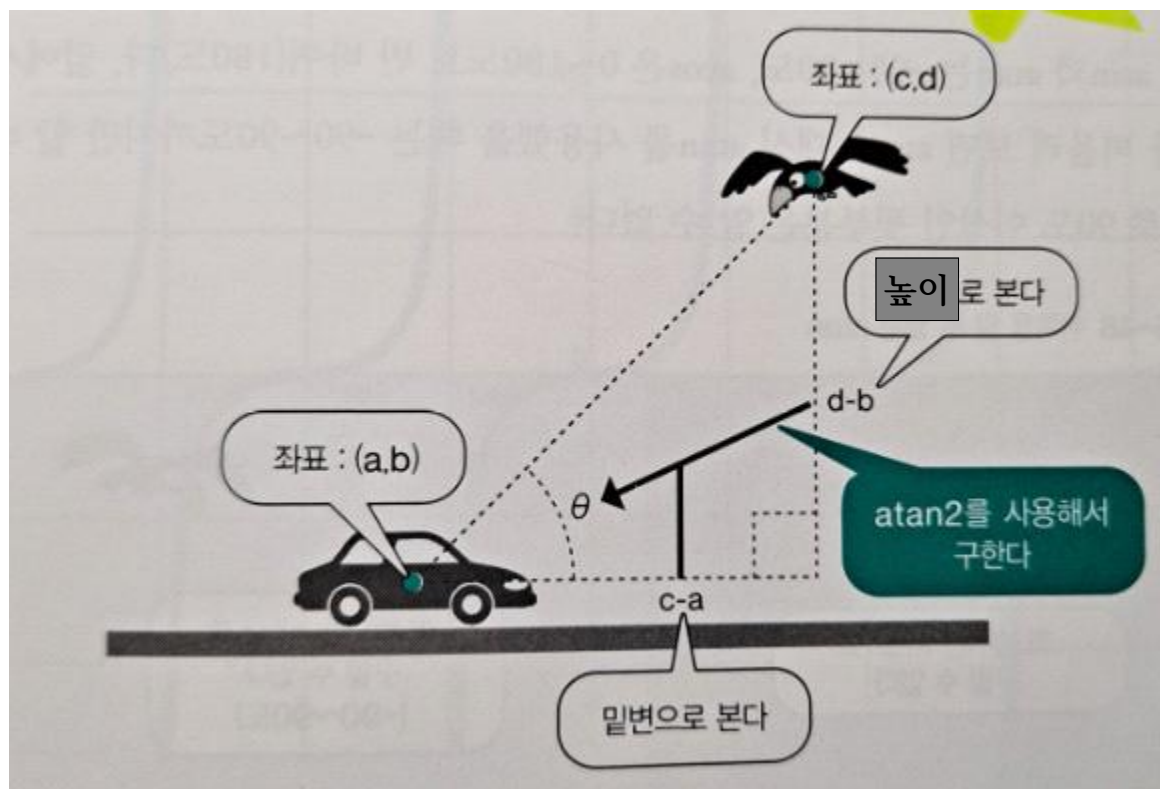
- 예전부터 자주 보다 atan OR atan2가 역삼각함수의 종류이며 asin , acos , atan 보다는 **atan2를 많이** 사용합니다.

```
Debug.Log(Mathf.Atan2(1, 1) * Mathf.Rad2Deg);
```



- 매개변수\_1은 밑변 , 매개변수\_2는 높이를 나타내며 , 그 도형의 tan값을 라디안 표기법으로 나타냅니다.

- 라디안 표기법에 Mathf.Rad2Deg 상수를 곱하여 각도 표기로 나타내어 줍니다.



### <3강\_numpy>

- 선형대수학에서의 어려운 계산을 sw(python의 numpy)를 통해 쉽게 계산할 수 있습니다.

```
import numpy as np

#1. 행렬 만들기
arr = np.mat([[1,2],[3,4]])

#2. 연산은 그대로 + , - , * , / 이용

#3. 역행렬(대소문자 구분함)
print(arr.I)

#4. 전치행렬(대소문자 구분함)
print(arr.T)

#5. 역행렬 계산
#A * A의 역행렬 = I지만, 계산결과가 제대로 나오지않으므로
#A * A의 역행렬 * A = A를 이용해서 검사합니다,
print(arr*arr.I*arr)

#6. 단위행렬 만들기 ex : 행렬 크기가 2
b = np.eye(2)
print(b)

#7. 대각합
c = np.trace(arr)
print(c)
```

- 실전에서는 어떤 벡터와 행렬(변환)을 곱해서 새로운 벡터를 만들어냅니다. 행렬은 np.mat를 사용하면 되고 벡터도 지금은 np.mat으로 좌표를 나타낼 수 있지만 엄밀하게 벡터를 무엇으로 나타내는지 아직은 모릅니다.

- A와 A의 Transpose의 곱이 I라면 A는 직교행렬 입니다.

## <4강\_변환>

### 1. 변환(Transformation)

#### 정의

입력과 출력이 모두 벡터인 함수를 **변환(Transformation)**이라 한다. 그리고  $\mathbb{R}^n$ 에서  $\mathbb{R}^m$ 으로의 변환  $T : \mathbb{R}^n \rightarrow \mathbb{R}^m$ 에서  $\mathbf{w} = T(\mathbf{x})$ 를 벡터  $\mathbf{x}$ 의  $T$ 에 대한 **이미지(image)**,  $\mathbf{x}$ 를 벡터  $\mathbf{w}$ 의 **원상(pre-image)**이라 한다.



변환의 특수한 경우로,  $A$ 가  $m \times n$  행렬이고,  $T_A(\mathbf{x}) = A\mathbf{x}$ ,  $\mathbf{x} \in \mathbb{R}^n$ 인  $T_A : \mathbb{R}^n \rightarrow \mathbb{R}^m$ 를 **행렬변환(matrix transformation)**이라 한다.

- 다른 차원으로도 변환이 가능합니다.

#### <변환의 두 가지 표기법>

① 행렬 사용 X

$$T\left(\begin{bmatrix} x \\ y \end{bmatrix}\right) = \begin{bmatrix} x \\ y+1 \end{bmatrix}$$

② 행렬을 사용하여 표기하고 해당 행렬이 변환을 의미합니다.

$$T\left(\begin{bmatrix} x \\ y \\ z \end{bmatrix}\right) = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & -1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

#### <Example>

**예제 1**에서 주어진 선형변환  $T$ 는  $T\left(\begin{bmatrix} x \\ y \end{bmatrix}\right) = \begin{bmatrix} x \\ y \\ x-y \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$ 이므로

$T$ 는 행렬  $A = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & -1 \end{bmatrix}$ 에 대한 행렬변환이다.

## 2. 선형 변환 (Linear Transformation)

### 정의

$\mathbb{R}^n$ 에서  $\mathbb{R}^m$ 으로의 변환  $T: \mathbb{R}^n \rightarrow \mathbb{R}^m$ 가 임의의 벡터  $u, v \in \mathbb{R}^n$ 와 임의의 스칼라  $k$ 에 대하여 다음 두 조건을 만족하면  $T$ 를  $\mathbb{R}^n$ 에서  $\mathbb{R}^m$ 로의 **선형변환(linear transformation)**이라고 한다.

$$(1) \quad T(u+v) = T(u) + T(v) \qquad (2) \quad T(ku) = kT(u) \quad (k \in \mathbb{R})$$

특히,  $\mathbb{R}^n$ 에서  $\mathbb{R}^n$  자신으로의 선형변환  $T: \mathbb{R}^n \rightarrow \mathbb{R}^n$ 를  $\mathbb{R}^n$  위의 **선형연산자(linear operator)**라고 한다.

- linear transformation은 많은 성질을 만족하기 때문에 매우 중요한 개념입니다.

✉ 예제

$\mathbf{u}_1 = (1, 1, 0)$ ,  $\mathbf{u}_2 = (1, 0, 1)$ ,  $\mathbf{u}_3 = (0, 1, 1)$ 인  $\mathbb{R}^3$ 의 한 기저  $\{\mathbf{u}_1, \mathbf{u}_2, \mathbf{u}_3\}$ 가 있다. 이때 선형변환  $T: \mathbb{R}^3 \rightarrow \mathbb{R}^2$ 을

$$T(\mathbf{u}_1) = (2, -3), \quad T(\mathbf{u}_2) = (-4, 6), \quad T(\mathbf{u}_3) = (-2, 4)$$

로 정의한다.  $T((3, 5, -4))$ 를 구하여라.

$$\mathbf{u} = \alpha_1 \mathbf{u}_1 + \alpha_2 \mathbf{u}_2 + \alpha_3 \mathbf{u}_3$$

즉  $(3, 5, -4) = \alpha_1(1, 1, 0) + \alpha_2(1, 0, 1) + \alpha_3(0, 1, 1)$

에서

$$\begin{cases} \alpha_1 + \alpha_2 = 3 \\ \alpha_1 + \alpha_3 = 5 \\ \alpha_2 + \alpha_3 = -4 \end{cases}$$

$$\therefore \alpha_1 = 6, \quad \alpha_2 = -3, \quad \alpha_3 = -1$$

$$\therefore \mathbf{u} = (3, 5, -4) = 6\mathbf{u}_1 - 3\mathbf{u}_2 - \mathbf{u}_3$$

$$\therefore T(\mathbf{u}) = T((3, 5, -4)) = 6T(\mathbf{u}_1) - 3T(\mathbf{u}_2) - T(\mathbf{u}_3)$$

$$= 6(2, -3) - 3(-4, 6) - (-2, 4)$$

$$= (26, -40)$$



- 선형 변환의 2가지 정의를 이용하면 쉽게 해결할 수 있습니다.
- 식을 세워 3차 연립방정식을 해결하면 계수를 구할 수 있고,  $6*\mathbf{u}_1 + -3*\mathbf{u}_2 + -1*\mathbf{u}_3$ 을 계산하면 됩니다.

### 3. 특수한 선형 변환

#### \* 참고 특수한 선형변환

- 영변환(zero transformation): 임의의  $\mathbf{v} \in \mathbb{R}^n$ 에 대하여  $T: \mathbb{R}^n \rightarrow \mathbb{R}^m$ 을  $T(\mathbf{v}) = \mathbf{0}$ 으로 정의하면  $T$ 는 선형변환이 된다. 이를 영변환이라 한다.
- 항등변환(identity transformation): 임의의  $\mathbf{v} \in \mathbb{R}^n$ 에 대하여  $T: \mathbb{R}^n \rightarrow \mathbb{R}^n$ 을  $T(\mathbf{v}) = \mathbf{v}$ 로 정의하면  $T$ 는 선형변환이 된다. 이를 항등변환이라 한다.
- 행렬변환(matrix transformation):  $A$ 가  $m \times n$  행렬일 때,  $\mathbb{R}^n$ 의 임의의 벡터  $\mathbf{x}$ 에 대하여  $T_A(\mathbf{x}) = A\mathbf{x}$ 라 정의하면  $T_A$ 는  $\mathbb{R}^n$ 에서  $\mathbb{R}^m$ 으로의 선형변환이 된다. 이를 행렬변환이라 한다.

- 모든 선형 변환은 그에 대한 행렬 변환이 반드시 존재합니다. (행렬 문제로 풀 수 있습니다.)

예제

6

선형변환  $T : \mathbb{R}^3 \rightarrow \mathbb{R}^4$ 이  $T\left(\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}\right) = \begin{bmatrix} x_1 + 2x_2 \\ -x_1 - x_2 \\ x_3 \\ x_1 + x_3 \end{bmatrix}$  일 때,  $T$ 의 표준행렬을 이용하여

$T(\mathbf{x}) = A\mathbf{x}$ 으로 표시하여라.

풀이

행렬을  $A = \begin{bmatrix} 1 & 2 & 0 \\ -1 & -1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 1 \end{bmatrix}$ 라 놓으면,  $T(\mathbf{x}) = A\mathbf{x}$ 인 행렬변환이 된다.

$$T(\mathbf{x}) = T\left(\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}\right) = \begin{bmatrix} x_1 + 2x_2 \\ -x_1 - x_2 \\ x_3 \\ x_1 + x_3 \end{bmatrix} = \begin{bmatrix} 1 & 2 & 0 \\ -1 & -1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = A\mathbf{x}$$

□

- 선형 변환을 표준 행렬을 이용하여 행렬 변환을 나타내려면 Input과 Output의 행렬 크기를 확인해야 합니다.
- Input은 3차원이므로  $3 \times 1$  이고 , Output은 4차원이므로  $4 \times 1$ 입니다. 그러므로 행렬 A는  $4 \times 3$  행렬을 만족합니다.

#### 4. 선형 변환의 기하학적 의미

##### 1) 회전변환

- (1)  $R_\theta : \mathbb{R}^2 \rightarrow \mathbb{R}^2$ 는  $\mathbb{R}^2$  안의 벡터를 원점을 중심으로  $\theta$  만큼 시계 반대방향으로 회전하는 선형변환이다.

$$[R_\theta] = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

<유도 과정>

##### 2) 정사영

- (2)  $x$ -축상의 정사영(orthogonal projection)은  $P : \mathbb{R}^2 \rightarrow \mathbb{R}^2$ 인 선형변환이다.

$$P(\mathbf{x}) = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} x \\ 0 \end{bmatrix}$$

-  $x$ 축 OR  $y$ 축의 값이 0이 되면 정사영입니다.

##### 3) 축 변환

$$T(\mathbf{x}) = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} x \\ -y \end{bmatrix}$$

- 변환 하고자 하는 축을 기준으로 양과 음을 뒤집으면 됩니다.

## 5. 핵과 치역

### 정의

#### 선형변환의 핵과 치역

벡터공간  $V$ ,  $W$ 에 대하여  $T : V \rightarrow W$ 은 선형변환이다.

- (1)  $T(\mathbf{v}) = \mathbf{0}$ 가 되는  $V$ 의 벡터  $\mathbf{v}$ 의 집합을  $T$ 의 핵(kernel), 또는 퇴화공간(null space)이라 하고 다음과 같이 나타낸다.

$$\ker T = \{\mathbf{v} \mid T(\mathbf{v}) = \mathbf{0}, \mathbf{v} \in V\}$$

- (2)  $T(\mathbf{v}) = \mathbf{w}$ 가 되는  $W$ 의 벡터  $\mathbf{w}$ 의 집합을  $T$ 의 치역(range) 또는 치역공간(image space)이라 하고 다음과 같이 나타낸다. 이때  $\mathbf{v} \in V$

$$\text{im } T = \{\mathbf{w} \mid T(\mathbf{v}) = \mathbf{w}, \mathbf{v} \in V, \mathbf{w} \in W\}$$

### 1) Null Space (핵)

= 벡터 공간  $V$ 에서 선형 변환을 한 결과가 0벡터인 벡터의 집합( $V$ 에 존재하는)을 Null Space라고 부르며 '핵'이라고도 합니다.

선형변환  $T : \mathbb{R}^2 \rightarrow \mathbb{R}^2$ ,  $T(x, y) = (x - y, 0)$ 에 대하여  $\ker T$ 를 구하여라.

### 풀이

$$\ker T = \{(x, y) \in \mathbb{R}^2 \mid (x - y, 0) = (0, 0)\} = \{(x, y) \in \mathbb{R}^2 \mid y = x\}$$

- 변환이  $x-y$ 일 때  $x=y$ 라면 0을 만족하므로  $x=y$ 인 벡터들이 Null Space를 구성합니다.

2) 치역

= 벡터 공간  $V$ 에서 선형 변환을 한 결과가 문제에서 요구하는 벡터인 벡터의 집합(결과 벡터 공간에 존재하는)을 치역이라고 합니다.

6. 퇴화차수와 계수

정의

$T : V \rightarrow W$ 가 선형변환이다. 이때

- (1)  $T$ 의 핵의 차원을  $T$ 의 퇴화차수(nullity)라 하는데 이것은  $T$ 의 핵공간의 차원을 의미하며 다음과 같이 나타낸다.

$$n(T) = \dim(\ker T)$$

- (2)  $T$ 의 치역의 차원을  $T$ 의 계수(rank)라 하고 다음과 같이 나타낸다.

$$r(T) = \dim(\operatorname{im} T)$$

1) 퇴화차수

= 핵의 차원

- 0 벡터는 0차원 입니다.

2) 계수

= 치역의 차원

### 3. 표준 기저

기저벡터 중에서도 원소 중 하나만 값이 1이고 다른 값은 0으로 이루어진 다음과 같은 기저벡터를 표준기저벡터라고 한다.

## <강\_행렬>

- 행렬은 선형연립방정식을 풀 때뿐만 아니라 소리와 이미지를 디지털로 전송하는 도구로도 이용됩니다. 우리는 행렬 사이의 덧셈과 곱셈 같은 연산을 정의합니다. 이러한 행렬 연산은 다양한 선형연립방정식을 쉽게 푸는 도구도 됩니다. 또 행렬의 곱셈은 합성함수를 다룰 때 훌륭한 도구가 됩니다.

- 3-1강은 기초적인 행렬에 대해서 학습합니다.

- 3-2강은 역행렬에 대해 학습하며, 개인적인 생각에  $\mathbb{R}^2$ 의 벡터에 대해, 변환(T)을 하여 새로운 벡터가 나왔을 때, 새로운 벡터와 변환을 알고 변환 이전의 벡터를 알지 못할 때, 양변에 변환의 역행렬을 곱하여 구할 수 있습니다.

- 3-3강은  $3 \times 3$  이상의 행렬에 대해 역행렬을 구하는 방식인데, PYTHON의 numpy를 이용하여 계산하면 간단하게 구할 수 있습니다.

## 2. space (공간)

### 1) Subspace

집합  $W (= \emptyset)$ 가  $\mathbb{R}^n$ 의 부분집합이라 하자. 이때 다음 두 조건을 만족하면  $W$ 는  $\mathbb{R}^n$ 의 부분공간(subspace)이다.

(1)  $x, y \in W \Rightarrow x + y \in W$  (덧셈에 닫혀 있다.)

(2)  $x \in W, k \in \mathbb{R} \Rightarrow kx \in W$  (스칼라배에 닫혀 있다.)

- 집합  $W$ 가 (1)과 (2)조건을 만족시키면  $W$ 는  $\mathbb{R}^n$ 의 Subspace 입니다.

-  $\mathbb{R}^n$ 의 모든 subspace는 영벡터를 포함해야 합니다.

-  $y = x+1$ 은  $\mathbb{R}^2$ 의 subspace가 될 수 없습니다. 왜냐하면 (0,1)과 (1,2)를 더한 (1,3)은 직선에 포함되지 않습니다.

## 2) Null Space

행렬  $A = [a_{ij}]_{m \times n}$  에 대하여 집합  $W = \{\mathbf{x} \in \mathbb{R}^n \mid A\mathbf{x} = \mathbf{0}\}$  은  $n$  차원공간  $\mathbb{R}^n$  의 부분공간임을 보여라(이러한  $W$ 를  $A\mathbf{x} = \mathbf{0}$ 의 **해공간(solution space)** 또는  $A$ 의 **영공간(null space)**이라 한다).

**풀이** 우선  $A\mathbf{0} = \mathbf{0}$ 이므로  $\mathbf{0} \in W$ 이다. 따라서  $W \neq \emptyset$  이다.

이제  $\mathbf{x}, \mathbf{y} \in W$ 와  $k \in \mathbb{R}$ 에 대하여  $A\mathbf{x} = \mathbf{0}$ ,  $A\mathbf{y} = \mathbf{0}$ 이므로

$$A(\mathbf{x} + \mathbf{y}) = A\mathbf{x} + A\mathbf{y} = \mathbf{0} + \mathbf{0} = \mathbf{0},$$

$$A(k\mathbf{x}) = k(A\mathbf{x}) = k\mathbf{0} = \mathbf{0}$$

$$\therefore \mathbf{x} + \mathbf{y} \in W, \quad k\mathbf{x} \in W$$

따라서  $W$ 는  $\mathbb{R}^n$ 의 부분공간이다. ■

## 3. Linear Combination (일차 결합)

-

## <5강\_행렬식>

\*\*\* Bio에서 필기했으니까 가져 오자 \*\*\*